

Planejamento Lógico TaskTracker

Thiago Castro Barreto

1. Apresentação e Entendimento do Problema

1.1 Identificação

- Nome completo: Thiago Castro Barreto
- Curso: Ciências de Dados
- Polo/Turma: Bootcamp 2
- E-mail institucional: thiago.cb@sempreceub.com

1.2 O Problema a Ser Resolvido

No dia a dia, é extremamente comum que pessoas e equipes precisem gerenciar simultaneamente diversas tarefas de origens distintas, tais como compromissos profissionais, entregas acadêmicas, rotinas de estudos e afazeres pessoais. O grande obstáculo surge quando esse gerenciamento é feito de maneira desestruturada — confiando unicamente na memória, em anotações físicas espalhadas ou em trocas de mensagens informais via aplicativos de conversa (como o WhatsApp).

Esse cenário sem centralização gera consequências diretas e prejudiciais:

- **Perda de prazos e esquecimentos:** Sem um lembrete visual central, compromissos críticos são esquecidos ou entregues com atraso.
- **Falta de clareza nas prioridades:** Dificuldade em diferenciar atividades urgentes e de alto impacto de tarefas secundárias, levando à alocação incorreta de tempo e energia.
- **Desconhecimento do progresso:** Incapacidade de mensurar rapidamente o que já foi finalizado e o que ainda demanda atenção.
- **Ineficiência e estresse:** O esforço cognitivo constante para tentar manter o controle das pendências gera retrabalho, ansiedade e queda contínua na produtividade individual ou do grupo.

1.3 A Solução Proposta (TaskTracker)

O **TaskTracker** é um sistema centralizado e intuitivo de gerenciamento de tarefas desenvolvido para estruturar e simplificar a rotina do usuário. A aplicação centraliza todo o fluxo de atividades em uma única plataforma organizada.

Principais recursos e funcionamento do sistema:

- **Cadastro Completo de Tarefas:** Permite o registro detalhado de cada atividade informando um Título (obrigatório), uma Descrição (detalhando o que deve ser feito), o Nível de Prioridade (Alta, Média ou Baixa) e uma Data Limite opcional para acompanhamento do prazo.
- **Acompanhamento de Status:** Cada tarefa possui um ciclo de vida claro, iniciando como "Pendente" e podendo ser alterada para "Concluída" conforme a evolução do trabalho.
- **Visualização e Métricas de Progresso:** O usuário pode consultar a lista completa de tarefas organizadas, identificando com facilidade as pendências e recebendo um balanço em tempo real do total de tarefas concluídas e pendentes.
- **Feedbacks Claros e Orientativos:** A cada interação (cadastro, atualização ou validação de erros), o sistema emite mensagens explicativas para guiar o usuário com segurança.

2. Decomposição do Problema e Dados do Sistema

2.1 Entradas de Dados

- **Título da tarefa:** Texto descritivo principal que identifica a atividade (campo obrigatório; não pode ser salvo em branco).
- **Descrição da tarefa:** Texto detalhado opcional explicando o contexto, passos ou observações para a execução da tarefa.
- **Prioridade:** Classificação da urgência da tarefa, restrita estritamente às opções *Alta*, *Média* ou *Baixa* (campo obrigatório).
- **Data limite:** Campo de data opcional para registrar o prazo final de entrega no formato padrão (AAAA-MM-DD).

2.2 Saídas de Dados

- **Lista organizada de tarefas:** Exibição clara e ordenada das tarefas cadastradas, estruturadas prioritariamente pelo nível de urgência (Alta, Média, Baixa) e pelo status.
- **Mensagens de confirmação e alerta:** Notificações de feedback do sistema indicando o sucesso das operações (ex: "Tarefa cadastrada com sucesso", "Status alterado para concluída") ou avisos de validação (ex: "Título é obrigatório", "Prioridade inválida").
- **Resumo estatístico de produtividade:** Exibição dos indicadores numéricos ao final da consulta, contendo:
 - Total de tarefas pendentes;
 - Total de tarefas concluídas;
 - Total geral de tarefas registradas.

2.3 Regras do Sistema

1. **Obrigatoriedade do Título:** O sistema bloqueia expressamente o cadastro de qualquer tarefa que não possua título ou cujo título contenha apenas espaços vazios.
2. **Restrição de Prioridade:** O sistema valida o campo de prioridade e aceita única e exclusivamente os valores "Alta", "Média" ou "Baixa". Qualquer valor divergente deve ser rejeitado com mensagem de instrução.
3. **Validação de Existência para Conclusão:** Uma tarefa só pode ser marcada como "Concluída" se ela já estiver devidamente cadastrada na lista e puder ser identificada no sistema.

3. Receita Lógica do Sistema (Algoritmo em Português)

3.1 Passo a Passo do Cadastro

1. Pedir ao usuário que digite o título da tarefa.
2. Verificar se o título está vazio ou contém apenas espaços em branco:
 - Se estiver vazio, exibir uma mensagem de erro orientando o usuário e pedir o título novamente até que um valor válido seja informado.
3. Pedir ao usuário a descrição da tarefa (campo opcional).
4. Pedir ao usuário o nível de prioridade (Alta, Média ou Baixa):
 - Verificar se a opção informada é válida; caso não seja, exibir mensagem de alerta e solicitar a prioridade novamente.
5. Pedir ao usuário a data limite da tarefa (opcional).

6. Gravar a nova tarefa na lista de tarefas cadastradas com o status inicial igual a "Pendente".
7. Exibir uma mensagem de confirmação informando ao usuário que a tarefa foi cadastrada com sucesso.

3.2 Passo a Passo da Visualização

1. Acessar o repositório/lista de tarefas cadastradas.
2. Verificar se a lista está vazia:
 - Se estiver vazia, exibir a mensagem ao usuário: "Não há tarefas cadastradas no momento."
3. Se houver tarefas cadastradas na lista:
 - Ordenar a exibição dando preferência para a prioridade (exibir primeiro as de prioridade Alta, depois Média e por fim Baixa).
 - Percorrer e exibir cada tarefa individualmente, mostrando claramente: Título, Descrição, Prioridade, Data Limite (se houver) e Status (Pendente ou Concluída).
4. Calcular e contabilizar a quantidade total de tarefas com status "Pendente" e com status "Concluída".
5. Exibir ao final da lista o resumo contendo o Total de Tarefas Pendentes, Total de Tarefas Concluídas e Total Geral.

Mapeamento Arquitetural para Git e GitHub

Planejamento da estrutura do repositório do TaskTracker — Preparação para a Fase 2

Nome: Thiago Castro Barreto

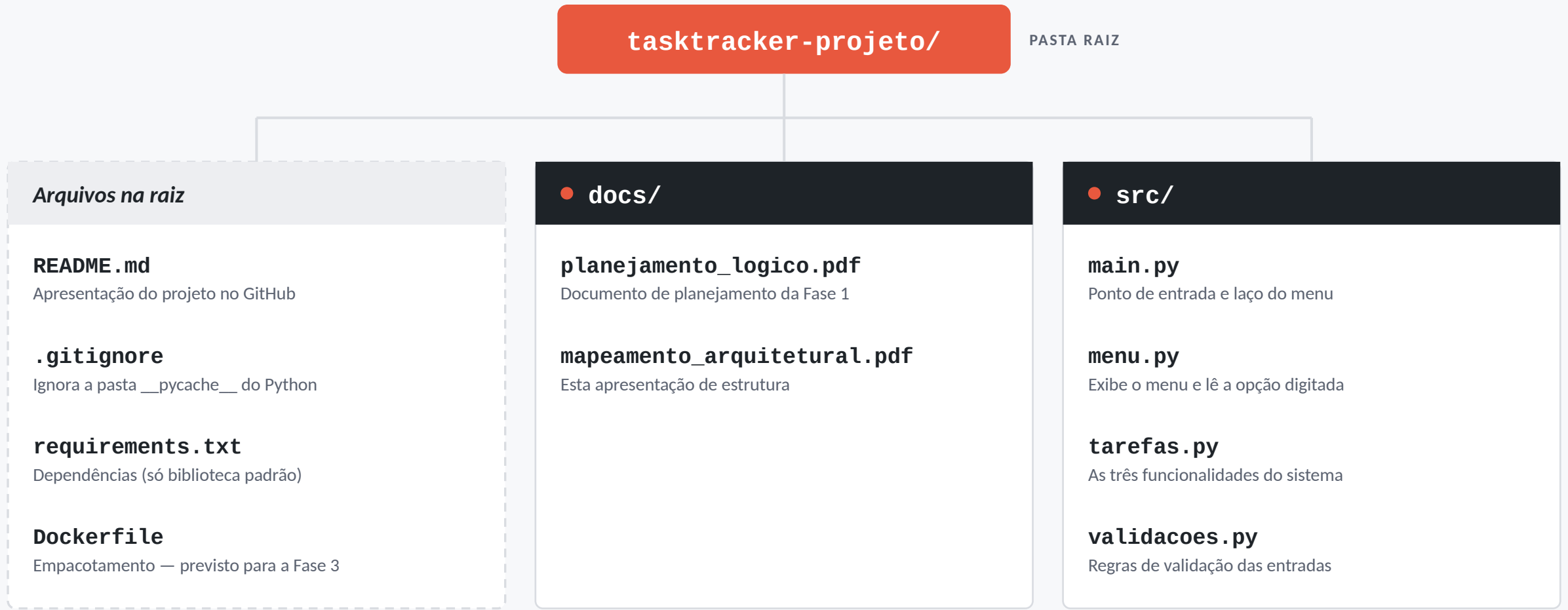
Curso: Ciências de Dados

Polo/Turma: Bootcamp 2

Email: thiago.cb@sempreceub.com

Desenho da Árvore de Repositório

Estrutura de pastas e arquivos do TaskTracker que será versionada no GitHub



Observação Importante: O bloco tracejado não é uma pasta, ele reúne os arquivos que ficam soltos na raiz do repositório, no mesmo nível de docs/ e src/.

Arquivos da Raiz e Documentação

Finalidade de cada item que fica fora da pasta de código-fonte

- **README.md**

Apresenta o projeto na página inicial do GitHub: o que é o TaskTracker, suas funcionalidades e como executá-lo.

- **.gitignore**

Informa ao Git quais arquivos não devem ir para o repositório, como a pasta `__pycache__` gerada pelo Python.

- **requirements.txt**

Lista as dependências do projeto. Nesta fase o TaskTracker usa apenas a biblioteca padrão do Python.

- **Dockerfile**

Receita de empacotamento do programa em container, prevista para a Fase 3 do projeto.

- **docs/**

Reúne a documentação do projeto, mantendo-a separada do código-fonte.

- **docs/planejamento_logico.pdf**

Documento de planejamento lógico da Fase 1, guardado no repositório como registro do projeto.

- **docs/mapeamento_arquitetural.pdf**

Esta apresentação, com o diagrama de hierarquia planejado para o repositório.

- **src/**

Reúne o código-fonte (source) do projeto, seguindo a convenção usada em projetos Python.

Divisão do Código-Fonte em src/

Cada módulo corresponde a uma parte da receita lógica descrita no documento

`main.py`

Ponto de entrada do programa

- Cria a lista vazia que guarda as tarefas cadastradas
- Controla o laço principal do menu de opções
- Encerra o programa quando o usuário solicita a saída

`menu.py`

Interface com o usuário

- Exibe as opções disponíveis: cadastrar, visualizar, concluir e sair
- Lê a opção digitada e encaminha para a função correspondente
- Avisa o usuário quando a opção digitada é inválida

`tarefas.py`

As três funcionalidades

- `cadastrar_tarefa()` — registra título, descrição, prioridade e data limite (Item 3.1)
- `listar_tarefas()` — exibe as tarefas ordenadas por prioridade e o resumo estatístico (Item 3.2)
- `concluir_tarefa()` — altera o status de “Pendente” para “Concluída”

`validacoes.py`

Regras de validação das entradas

- Título não pode ficar em branco ou conter apenas espaços (Regra 2.3)
- Prioridade só pode ser Alta, Média ou Baixa (Regra 2.3)
- Tarefa só pode ser concluída se já estiver cadastrada na lista (Regra 2.3)

Plano de Aprendizizado para Git e GitHub

O que espero aprender sobre controle de versão na Fase 2 do projeto

01

Commits

Registrar cada alteração de forma organizada, com mensagens claras, para que o histórico conte a evolução do TaskTracker.

02

Branches

Desenvolver e testar cada módulo (main, menu, tarefas, validações) separadamente, sem comprometer a versão que já funciona.

03

Conflitos e histórico

Resolver conflitos de código e navegar pelo histórico para recuperar versões anteriores caso algo quebre.

04

Publicação no GitHub

Publicar o repositório de forma profissional, com README bem escrito e estrutura de pastas coerente.